

Common C# code conventions

Coding conventions are essential for maintaining code readability, consistency, and collaboration within a development team. Code that follows industry practices and established guidelines is easier to understand, maintain, and extend. Most projects enforce a consistent style through code conventions. The [dotnet/docs](#) and [dotnet/samples](#) projects are no exception. In this series of articles, you learn our coding conventions and the tools we use to enforce them. You can take our conventions as-is, or modify them to suit your team's needs.

We chose our conventions based on the following goals:

1. *Correctness*: Our samples are copied and pasted into your applications. We expect that, so we need to make code that's resilient and correct, even after multiple edits.
2. *Teaching*: The purpose of our samples is to teach all of .NET and C#. For that reason, we don't place restrictions on any language feature or API. Instead, those samples teach when a feature is a good choice.
3. *Consistency*: Readers expect a consistent experience across our content. All samples should conform to the same style.
4. *Adoption*: We aggressively update our samples to use new language features. That practice raises awareness of new features, and makes them more familiar to all C# developers.

Important

These guidelines are used by Microsoft to develop samples and documentation. They were adopted from the [.NET Runtime, C# Coding Style](#) and [C# compiler \(roslyn\)](#) guidelines. We chose those guidelines because of their adoption over several years of Open Source development. These guidelines help community members participate in the runtime and compiler projects. They're meant to be an example of common C# conventions, and not an authoritative list (see [Framework Design Guidelines](#) for detailed guidelines).

The *teaching* and *adoption* goals are why the docs coding convention differs from the runtime and compiler conventions. Both the runtime and compiler have strict performance metrics for hot paths. Many other applications don't. Our *teaching* goal mandates that we don't prohibit any construct. Instead, samples show when constructs

should be used. We update samples more aggressively than most production applications do. Our *adoption* goal mandates that we show code you should write today, even when code written last year doesn't need changes.

This article explains our guidelines. The guidelines evolve over time, and you'll find samples that don't follow our guidelines. We welcome PRs that bring those samples into compliance, or issues that draw our attention to samples we should update. Our guidelines are Open Source and we welcome PRs and issues. However, if your submission would change these recommendations, open an issue for discussion first. You're welcome to use our guidelines, or adapt them to your needs.

Tools and analyzers

Tools can help your team enforce your conventions. You can enable [code analysis](#) to enforce the rules you prefer. You can also create an [editorconfig](#) so that Visual Studio automatically enforces your style guidelines. As a starting point, you can copy the [dotnet/docs.editorconfig](#) to use our style.

These tools make it easier for your team to adopt your preferred guidelines. Visual Studio applies the rules in all [.editorconfig](#) files in scope to format your code. You can use multiple configurations to enforce corporate-wide conventions, team conventions, and even granular project conventions.

Code analysis produces warnings and diagnostics when it detects rule violations. You configure the rules you want applied to your project. Then, each CI build notifies developers when they violate any of the rules.

Diagnostic IDs

- [Choose appropriate diagnostic IDs](#) when building your own analyzers

Language guidelines

The following sections describe practices that the .NET docs team follows to prepare code examples and samples. In general, follow these practices:

- Utilize modern language features and C# versions whenever possible.
- Avoid outdated language constructs.
- Only catch exceptions that can be properly handled; avoid catching general exceptions. For example, sample code shouldn't catch the [System.Exception](#) type without an exception filter.
- Use specific exception types to provide meaningful error messages.
- Use LINQ queries and methods for collection manipulation to improve code readability.
- Use asynchronous programming with `async` and `await` for I/O-bound operations.
- Be cautious of deadlocks and use [Task.ConfigureAwait](#) when appropriate.
- Use the language keywords for data types instead of the runtime types. For example, use `string` instead of [System.String](#), or `int` instead of [System.Int32](#). This recommendation includes using the types `nint` and `nuint`.
- Use `int` rather than unsigned types. The use of `int` is common throughout C#, and it's easier to interact with other libraries when you use `int`. Exceptions are for documentation specific to unsigned data types.
- Use `var` only when a reader can infer the type from the expression. Readers view our samples on the docs platform. They don't have hover or tool tips that display the type of variables.
- Write code with clarity and simplicity in mind.
- Avoid overly complex and convoluted code logic.

More specific guidelines follow.

String data

- Use [string interpolation](#) to concatenate short strings, as shown in the following code.

C#

```
string displayName = $"{nameList[n].LastName}, {nameList[n].First-
Name}";
```

- To append strings in loops, especially when you're working with large amounts of text, use a [System.Text.StringBuilder](#) object.

```
var phrase =  
"lalalalalalalalalalalalalalalalalalalalalalalalalalalal";  
var manyPhrases = new StringBuilder();  
for (var i = 0; i < 10000; i++)  
{  
    manyPhrases.Append(phrase);  
}  
//Console.WriteLine("tra" + manyPhrases);
```

- ## C#

```
var message = """
    This is a long message that spans across multiple lines.
    It uses raw string literals. This means we can
    also include characters like \n and \t without escaping them.
    """;
```

- ## C#

```
// Execute the queries.
Console.WriteLine("scoreQuery:");
foreach (var student in scoreQuery)
{
    Console.WriteLine($"{student.Last} Score: {student.score}");
}
```

C#

```
public record Person(string FirstName, string LastName);
```

- Use camel case for primary constructor parameters on class and struct types.
- Use required properties instead of constructors to force initialization of property values:

C#

```
public class LabelledContainer<T>(string label)
{
    public string Label { get; } = label;
    public required T Contents
    {
        get;
        init;
    }
}
```

Arrays and collections

- Use collection expressions to initialize all collection types:

C#

```
string[] vowels = [ "a", "e", "i", "o", "u" ];
```

Delegates

- Use `Func<>` and `Action<>` instead of defining delegate types. In a class, define the delegate method.

C#

```
Action<string> actionExample1 = x => Console.WriteLine($"x is: {x}");

Action<string, string> actionExample2 = (x, y) =>
    Console.WriteLine($"x is: {x}, y is {y}");

Func<string, int> funcExample1 = x => Convert.ToInt32(x);

Func<int, int, int> funcExample2 = (x, y) => x + y;
```

- Call the method using the signature defined by the `Func<>` or `Action<>` delegate.

C#

```
actionExample1("string for x");
actionExample2("string for x", "string for y");
Console.WriteLine($"The value is {funcExample1("1")}");
Console.WriteLine($"The sum is {funcExample2(1, 2)}");
```

- If you create instances of a delegate type, use the concise syntax. In a class, define the delegate type and a method that has a matching signature.

C#

```
public delegate void Del(string message);
public static void DelMethod(string str)
{
    Console.WriteLine($"DelMethod argument: {str}");
}
```

- Create an instance of the delegate type and call it. The following declaration shows the condensed syntax.

C#

```
Del exampleDel2 = DelMethod;
exampleDel2("Hey");
```

- The following declaration uses the full syntax.

C#

```
Del exampleDel1 = new Del(DelMethod);
exampleDel1("Hey");
```

try-catch and using statements in exception

handling

- Use a [try-catch](#) statement for most exception handling.

C#

```
static double ComputeDistance(double x1, double y1, double x2, double
y2)
{
    try
    {
        return Math.Sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 -
y2));
    }
    catch (System.ArithmeticException ex)
    {
        Console.WriteLine($"Arithmetic overflow or underflow: {ex}");
        throw;
    }
}
```

- Simplify your code by using the C# [using statement](#). If you have a [try-finally](#) statement in which the only code in the `finally` block is a call to the [Dispose](#) method, use a `using` statement instead.

In the following example, the `try-finally` statement only calls `Dispose` in the `finally` block.

C#

```
Font bodyStyle = new Font("Arial", 10.0f);
try
{
    byte charset = bodyStyle.GdiCharSet;
}
finally
{
    bodyStyle?.Dispose();
}
```

You can do the same thing with a `using` statement.

C#

```
using (Font arial = new Font("Arial", 10.0f))
{
    byte charset2 = arial.GdiCharSet;
}
```

Use the new [using syntax](#) that doesn't require braces:

C#

```
using Font normalStyle = new Font("Arial", 10.0f);
byte charset3 = normalStyle.GdiCharSet;
```

&& and || operators

- Use [&&](#) instead of [&](#) and [||](#) instead of [|](#) when you perform comparisons, as shown in the following example.

C#

```
Console.WriteLine("Enter a dividend: ");
int dividend = Convert.ToInt32(Console.ReadLine());

Console.WriteLine("Enter a divisor: ");
int divisor = Convert.ToInt32(Console.ReadLine());

if ((divisor != 0) && (dividend / divisor) is var result)
{
    Console.WriteLine($"Quotient: {result}");
}
else
{
    Console.WriteLine("Attempted division by 0 ends up here.");
}
```

If the divisor is 0, the second clause in the `if` statement would cause a run-time error. But the `&&` operator short-circuits when the first expression is false. That is, it doesn't evaluate the second expression. The `&` operator would evaluate both, resulting in a run-time error when divisor is 0.

new operator

- Use one of the concise forms of object instantiation when the variable type matches the object type, as shown in the following declarations. This form isn't valid when the variable is an interface type, or a base class of the runtime type.

C#

```
var firstExample = new ExampleClass();
```

C#

```
ExampleClass instance2 = new();
```

The preceding declarations are equivalent to the following declaration.

C#

```
ExampleClass secondExample = new ExampleClass();
```

- Use object initializers to simplify object creation, as shown in the following example.

C#

```
var thirdExample = new ExampleClass { Name = "Desktop", ID = 37414,
    Location = "Redmond", Age = 2.3 };
```

The following example sets the same properties as the preceding example but doesn't use initializers.

C#

```
var fourthExample = new ExampleClass();
fourthExample.Name = "Desktop";
fourthExample.ID = 37414;
fourthExample.Location = "Redmond";
fourthExample.Age = 2.3;
```

Event handling

- Use a lambda expression to define an event handler that you don't need to remove later:

C#

```
public Form2()
{
    this.Click += (s, e) =>
    {
        MessageBox.Show(
            ((MouseEventArgs)e).Location.ToString());
    };
}
```

The lambda expression shortens the following traditional definition.

C#

```
public Form1()
{
    this.Click += new EventHandler(Form1_Click);
}

void Form1_Click(object? sender, EventArgs e)
{
    MessageBox.Show(((MouseEventArgs)e).Location.ToString());
}
```

Static members

Call `static` members by using the class name: *ClassName.StaticMember*. This practice makes code more readable by making static access clear. Don't qualify a static member defined in a base class with the name of a derived class. While that code compiles, the code readability is misleading, and the code might break in the future if you add a static member with the same name to the derived class.

LINQ queries

- Use meaningful names for query variables. The following example uses `seattleCustomers` for customers who are located in Seattle.

C#

```
var seattleCustomers = from customer in Customers
                       where customer.City == "Seattle"
                       select customer.Name;
```

- Use aliases to make sure that property names of anonymous types are correctly capitalized, using Pascal casing.

C#

```
var localDistributors =
    from customer in Customers
    join distributor in Distributors on customer.City equals distributor.City
    select new { Customer = customer, Distributor = distributor };
```

- Rename properties when the property names in the result would be ambiguous. For example, if your query returns a customer name and a distributor name, instead of leaving them as a form of `Name` in the result, rename them to clarify `CustomerName` is the name of a customer, and `DistributorName` is the name of a distributor.

C#

```
var localDistributors2 =
    from customer in Customers
    join distributor in Distributors on customer.City equals distributor.City
    select new { CustomerName = customer.Name, DistributorName = distributor.Name };
```

- Use implicit typing in the declaration of query variables and range variables. This guidance on implicit typing in LINQ queries overrides the general rules for [implicitly typed local variables](#). LINQ queries often use projections that create anonymous types. Other query expressions create results with nested generic types. Implicit typed variables are often more readable.

C#

```
var seattleCustomers = from customer in Customers
                        where customer.City == "Seattle"
                        select customer.Name;
```

- Align query clauses under the `from` clause, as shown in the previous examples.
- Use `where` clauses before other query clauses to ensure that later query clauses operate on the reduced, filtered set of data.

C#

```
var seattleCustomers2 = from customer in Customers
                        where customer.City == "Seattle"
                        orderby customer.Name
                        select customer;
```

- Access inner collections with multiple `from` clauses instead of a `join` clause. For example, a collection of `Student` objects might each contain a collection of test scores. When the following query is executed, it returns each score that is over 90, along with the family name of the student who received the score.

C#

```
var scoreQuery = from student in students
                  from score in student.Scores
                  where score > 90
                  select new { Last = student.LastName, score };
```

Implicitly typed local variables

- Use `implicit typing` for local variables when the type of the variable is obvious from the right side of the assignment.

C#

```
var message = "This is clearly a string.";
var currentTemperature = 27;
```


collection's name shouldn't be solely relied upon for inferring the type of its elements.

The following example uses explicit typing in a `foreach` statement.

C#

```
foreach (char ch in laugh)
{
    if (ch == 'h')
    {
        Console.Write("H");
    }
    else
    {
        Console.Write(ch);
    }
}
Console.WriteLine();
```

- use implicit type for the result sequences in LINQ queries. The section on [LINQ](#) explains that many LINQ queries result in anonymous types where implicit types must be used. Other queries result in nested generic types where `var` is more readable.

ⓘ Note

Be careful not to accidentally change a type of an element of the iterable collection. For example, it's easy to switch from [System.Linq.IQueryable](#) to [System.Collections.IEnumerable](#) in a `foreach` statement, which changes the execution of a query.

Some of our samples explain the *natural type* of an expression. Those samples must use `var` so that the compiler picks the natural type. Even though those examples are less obvious, the use of `var` is required for the sample. The text should explain the behavior.

File scoped namespace declarations

Most code files declare a single namespace. Therefore, our examples should use the file scoped namespace declarations:

C#

```
namespace MySampleCode;
```

Place the using directives outside the namespace declaration

When a `using` directive is outside a namespace declaration, that imported namespace is its fully qualified name. The fully qualified name is clearer. When the `using` directive is inside the namespace, it could be either relative to that namespace, or its fully qualified name.

C#

```
using Azure;

namespace CoolStuff.AwesomeFeature
{
    public class Awesome
    {
        public void Stuff()
        {
            WaitUntil wait = WaitUntil.Completed;
            // ...
        }
    }
}
```

Assuming there's a reference (direct, or indirect) to the [WaitUntil](#) class.

Now, let's change it slightly:

C#

```
namespace CoolStuff.AwesomeFeature
{
    using Azure;

    public class Awesome
    {
        public void Stuff()
        {
            WaitUntil wait = WaitUntil.Completed;
```

```

    }
}
// ...

```

And it compiles today. And tomorrow. But then sometime next week the preceding (untouched) code fails with two errors:

Console

- error CS0246: The type or namespace name 'WaitUntil' could not be found (are you missing a using directive or an assembly reference?)
- error CS0103: The name 'WaitUntil' does not exist in the current context

One of the dependencies introduced this class in a namespace then ends with .Azure:

C#

```

namespace CoolStuff.Azure
{
    public class SecretsManagement
    {
        public string FetchFromKeyVault(string vaultId, string secretId) {
        return null; }
    }
}

```

A using directive placed inside a namespace is context-sensitive and complicates name resolution. In this example, it's the first namespace that it finds.

- CoolStuff.AwesomeFeature.Azure
- CoolStuff.Azure
- Azure

Adding a new namespace that matches either CoolStuff.Azure or CoolStuff.AwesomeFeature.Azure would match before the global Azure namespace. You could resolve it by adding the global:: modifier to the using declaration. However, it's easier to place using declarations outside the namespace instead.

C#


```
namespace CoolStuff.AwesomeFeature
{
    using global::Azure;

    public class Awesome
    {
        public void Stuff()
        {
            WaitUntil wait = WaitUntil.Completed;
            // ...
        }
    }
}
```

Style guidelines

In general, use the following format for code samples:

- Use four spaces for indentation. Don't use tab characters.
- Align code consistently to improve readability.
- Limit lines to 65 characters to enhance code readability on docs, especially on mobile screens.
- Improve clarity and user experience by breaking long statements into multiple lines.
- Use the "Allman" style for braces: open and closing brace its own new line. Braces line up with current indentation level.
- Line breaks should occur before binary operators, if necessary.

Comment style

- Use single-line comments (`//`) for brief explanations.
- Avoid multi-line comments (`/* */`) for longer explanations.
Comments in the code samples aren't localized. That means explanations embedded in the code aren't translated. Longer, explanatory text should be placed in the companion article, so that it can be localized.
- For describing methods, classes, fields, and all public members use [XML comments](#).
- Place the comment on a separate line, not at the end of a line of code.

- Begin comment text with an uppercase letter.
- End comment text with a period.
- Insert one space between the comment delimiter (//) and the comment text, as shown in the following example.

C#

```
// The following declaration creates a query. It does not run
// the query.
```

Layout conventions

Good layout uses formatting to emphasize the structure of your code and to make the code easier to read. Microsoft examples and samples conform to the following conventions:

- Use the default Code Editor settings (smart indenting, four-character indents, tabs saved as spaces). For more information, see [Options, Text Editor, C#, Formatting](#).
- Write only one statement per line.
- Write only one declaration per line.
- If continuation lines aren't indented automatically, indent them one tab stop (four spaces).
- Add at least one blank line between method definitions and property definitions.
- Use parentheses to make clauses in an expression apparent, as shown in the following code.

C#

```
if ((startX > endX) && (startX > previousX))
{
    // Take appropriate action.
}
```

Exceptions are when the sample explains operator or expression precedence.

Security

Follow the guidelines in [Secure Coding Guidelines](#).

Last updated on 01/18/2025